

SIMATS ENGINEERING ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO1: *Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems(BL:3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:25 Duration : 1 Hour

Simulation

Code of Conduct:

*I **J.Goutham Kumar Reddy(192311140)**_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

J.Goutham Kumar Reddy

SIMATS ENGINEERING ASSESSMENT TOOL

- 1 Design a **Non-Deterministic Finite Automaton (NBA)** for an **Automatic Door System** based on sensor inputs. An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: *Closed*, *Opening*, *Open*, and *Closing*. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the *Closed* state. When a person is detected (P), the system may non-deterministically transition from *Closed* to *Opening*. Once opened, the system moves to the *Open* state. If no person is detected (N), the door transitions to *Closing* and eventually returns to the *Closed* state. Nondeterminism occurs when the system may remain in the *Open* state even if a person is detected or may start closing if no detection occurs.
- 2 Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.
- 3 Design DFA using simulator to accept the string the end with ab over set {a,b} W= aaabab
- 4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings “c”, “bc”, and “bcaaa” over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string “c” should be accepted directly from the start state, while “bc” should pass through states representing “b” and then “c”. Similarly, “bcaaa” should be processed through a sequence of states that track each additional “a” after “bc”. Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.
- 5 Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either “a” or “b”. The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either “a” or “b”. After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of “a” and “b”, thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

SIMATS ENGINEERING

ASSESSMENT TOOL — ANSWERS

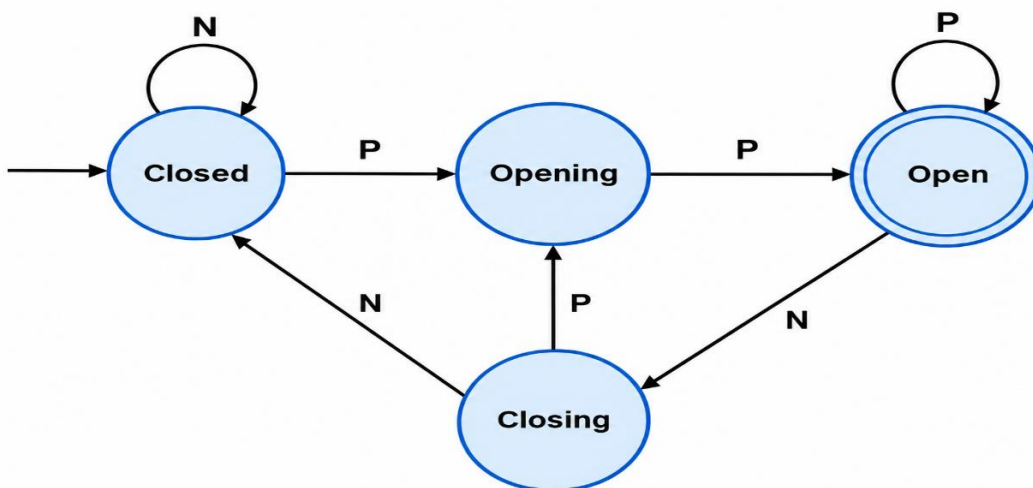
Answer to Question 1

The system is modeled as an NFA $N = (Q, \Sigma, \delta, q_0, F)$ where:

$Q = \{\text{Closed, Opening, Open, Closing}\}$ $\Sigma = \{P, N\}$ $q_0 = \text{Closed}$ $F = \{\text{Open}\}$ **Transition table:**

State	P	N
Closed (start)	{Opening}	{Closed}
Opening	{Open}	{Closed}
Open (accept)	{Open}	{Open, Closing}
Closing	{Opening}	{Closed}

State transition diagram:



Non-determinism: the essential non-determinism appears at state *Open* on input N, where the automaton may either stay in *Open* or move to *Closing* ($\delta(\text{Open}, N) = \{\text{Open}, \text{Closing}\}$), since the system need not close immediately even if no person is currently detected. Similarly from *Closing*, detecting a person sends control back to *Opening* without needing to fully close first, showing the system does not follow one single deterministic path for a given input sequence.

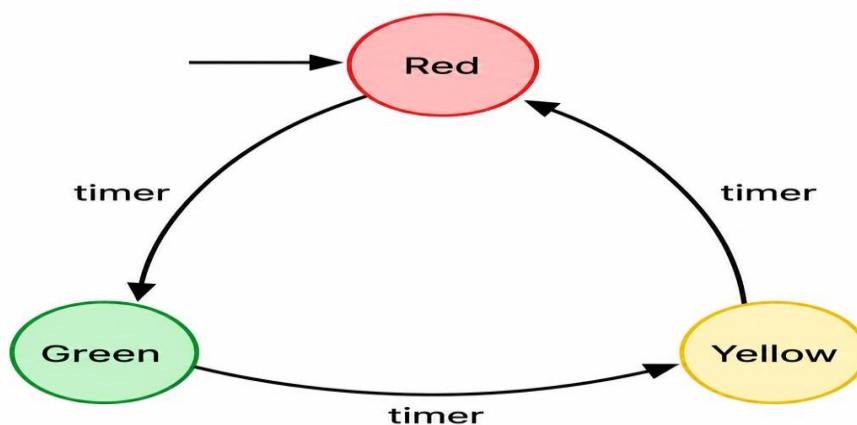
Answer to Question 2

The traffic light is modeled as a finite automaton (Moore-style, since each state itself is the output/signal shown to traffic): $M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{\text{Red, Green, Yellow}\}$ $\Sigma = \{\text{tick}\}$ (one timer-expiry signal) $q_0 = \text{Red}$ $F = Q$ (every state is a valid, reachable operating state — the automaton never 'terminates', it cycles continuously) **Transition table:**

Current State	Input (tick)	Next State	Typical Duration
Red	tick	Green	30 s
Green	tick	Yellow	25 s
Yellow	tick	Red	5 s

State transition diagram (cyclic):



Timing constraints:

Each state has an internal timer/counter that must expire before the 'tick' transition fires, so the automaton can be extended to a timed automaton where every state carries a duration guard (e.g., stay in Red for 30 s before transitioning). Since δ always maps back into $\{\text{Red, Green, Yellow}\}$ the automaton cycles forever, matching real intersection behaviour.

Answer to Question 3

Extensions:

Pedestrian crossing: add a state 'Walk' reached from Red (or inserted between Yellow and Red) with its own input symbol 'walk_request', after which the automaton returns to Red before resuming the normal cycle.

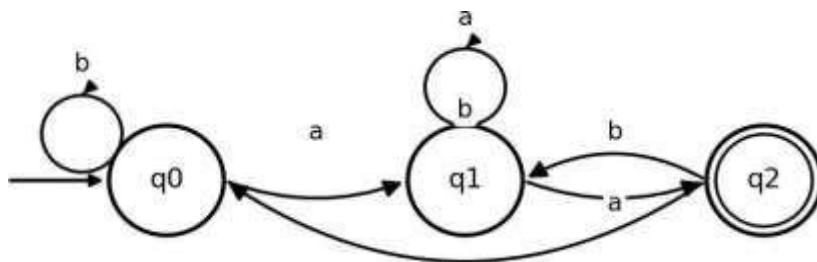
Emergency override: add an input symbol 'E' (emergency vehicle detected) that is enabled from every state and forces an immediate transition to Red (all-stop) regardless of the current timer, i.e. $\delta(\text{Green}, E) = \delta(\text{Yellow}, E) = \text{Red}$, after which the system resumes its normal cycle once E is withdrawn. This keeps the core model deterministic while layering safety logic on top.

DFA $M = (Q, \Sigma, \delta, q_0, F)$ where $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, start state q_0 , and $F = \{q_2\}$. q_0 = no useful suffix seen yet, q_1 = last symbol read was 'a', q_2 = last two symbols read were 'ab' (accepting).

Transition table (δ):

State	a	b
q_0 (start)	q_1	q_0
q_1	q_1	q_2
q_2 (accept)	q_1	q_0

State transition diagram:



Trace of $W = \text{aaabab}$: $q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_2$

$\xrightarrow{a} q_1 \xrightarrow{b} q_2$

The automaton ends in state **q_2** , which is a member of F . Since the last two symbols read were 'a' followed by 'b', the string **$W = \text{aaabab}$ is ACCEPTED.**

Since only three exact strings must be accepted, the DFA is built as a straight-line "prefix trie" with a single dead/trap state q_d absorbing every invalid continuation.

Answer to Question 4

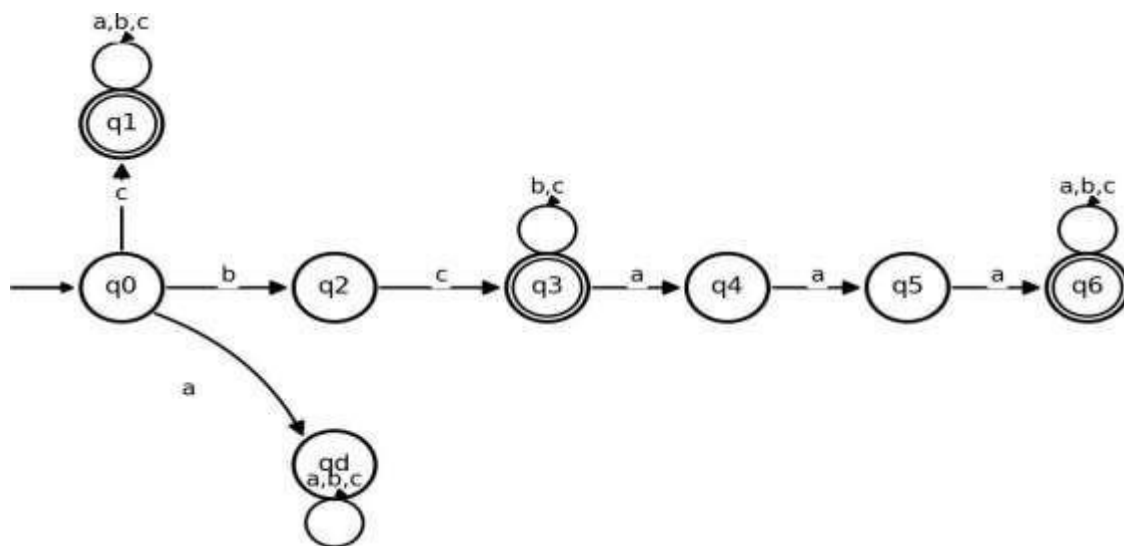
$M = (Q, \Sigma, \delta, q_0, F)$, $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_d\}$, $\Sigma = \{a, b, c\}$, start state q_0 , $F = \{q_1, q_3, q_6\}$ (accepting after exactly "c", "bc", and "bcaaa" respectively).

Transition table (δ) — every state fully defined over $\{a,b,c\}$:

State	a	b	c	Meaning
q_0 (start)	q_d	q_2	q_1	ε read so far
q_1 (accept)	q_d	q_d	q_d	read "c"
q_2	q_d	q_d	q_3	read "b"
q_3 (accept)	q_4	q_d	q_d	read "bc"
q_4	q_5	q_d	q_d	read "bca"
q_5	q_6	q_d	q_d	read "bcaa"
q_6 (accept)	q_d	q_d	q_d	read "bcaaa"
q_d (dead)	q_d	q_d	q_d	trap / reject

State transition diagram:

Answer to Question 5



Verification:

"c": $q_0 \xrightarrow{c} q_1$ — accepted. "bc": $q_0 \xrightarrow{b} q_2 \xrightarrow{c} q_3$ — accepted. "bcaaa": $q_0 \xrightarrow{b} q_2 \xrightarrow{c} q_3 \xrightarrow{a} q_4 \xrightarrow{a} q_5 \xrightarrow{a} q_6$ — accepted. Any other symbol at any point (e.g. "ca", "bca", "bcaaaa", "bb") diverts the machine into q_d , which is a non-accepting trap state with no way out, so the string is rejected. This can be implemented step-by-step in a simulator such as Autosim/JFLAP by adding these 8 states and 24 transitions exactly as listed above.

$M = (Q, \Sigma, \delta, q_0, F)$, $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_d\}$, $\Sigma = \{a, b\}$, start state q_0 , $F = \{q_1\}$.

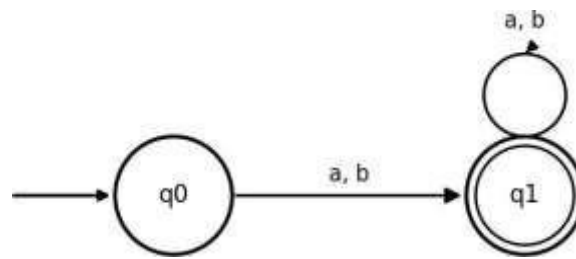
Since the alphabet contains only 'a' and 'b', every non-empty string automatically begins with one of these two symbols, so the DFA only needs to move to an accepting "trap-accept" state on the very first symbol and remain there for the rest of the string.

Transition table (δ):

State	a	b
q_0 (start, non-accepting)	q_1	q_1
q_1 (accept)	q_1	q_1

State transition diagram:

Answer to Question 6



Note on the dead/empty-string case:

q_0 itself is non-accepting, so the empty string ϵ (which begins with neither 'a' nor 'b') is correctly rejected. If the simulator requires an explicit trap for out-of-alphabet symbols, a separate dead state q_d can be added with $\delta(q_0, x) = \delta(q_1, x) = q_d$ for any x outside $\{a, b\}$; within this two-letter alphabet, however, q_0 and q_1 alone are sufficient — every string of length ≥ 1 is accepted, exactly as required.

SIMATS ENGINEERING ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and wellorganized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

